



Operations Research

Publication details, including instructions for authors and subscription information:
<http://pubsonline.informs.org>

Parallel Machine Scheduling by Column Generation

J. M. van den Akker, J. A. Hoogeveen, S. L. van de Velde,

To cite this article:

J. M. van den Akker, J. A. Hoogeveen, S. L. van de Velde, (1999) Parallel Machine Scheduling by Column Generation. Operations Research 47(6):862-872. <http://dx.doi.org/10.1287/opre.47.6.862>

Full terms and conditions of use: <http://pubsonline.informs.org/page/terms-and-conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact permissions@informs.org.

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

© 1999 INFORMS

Please scroll down for article—it is on subsequent pages



INFORMS is the largest professional society in the world for professionals in the fields of operations research, management science, and analytics.

For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

PARALLEL MACHINE SCHEDULING BY COLUMN GENERATION

J.M. VAN DEN AKKER

Department of Mathematical Models and Methods, National Aerospace Laboratory NLR, P.O. Box 90502, 1006 BM Amsterdam, The Netherlands, vdakker@nlr.nl

J.A. HOOGEVEEN

Department of Mathematics and Computing Science, Eindhoven University of Technology, P.O. Box 513, 5600 MB Eindhoven, The Netherlands, slam@win.tue.nl

S.L. VAN DE VELDE

Rotterdam School of Management, Erasmus University, P.O. Box 1738, 3000 DR Rotterdam, The Netherlands, s.velde@fac.fbk.eur.nl

(Received October 1995; revisions received April 1997, August 1997; accepted September 1998)

Parallel machine scheduling problems concern the scheduling of n jobs on m machines to minimize some function of the job completion times. If preemption is not allowed, then most problems are not only $\mathcal{NP}$ -hard, but also very hard from a practical point of view. In this paper, we show that strong and fast linear programming lower bounds can be computed for an important class of machine scheduling problems with additive objective functions. Characteristic of these problems is that on each machine the order of the jobs in the relevant part of the schedule is obtained through some priority rule. To that end, we formulate these parallel machine scheduling problems as a set covering problem with an exponential number of binary variables, n covering constraints, and a single side constraint. We show that the linear programming relaxation can be solved efficiently by column generation because the pricing problem is solvable in pseudo-polynomial time. We display this approach on the problem of minimizing total weighted completion time on m identical machines. Our computational results show that the lower bound is singularly strong and that the outcome of the linear program is often integral. Moreover, they show that our branch-and-bound algorithm that uses the linear programming lower bound outperforms the previously best algorithm.

1. INTRODUCTION

Parallel machine scheduling problems concern the scheduling of n jobs on m parallel machines to minimize some function of the job completion times. Problems in which preemption of jobs is not allowed decompose into two subproblems: *assigning* jobs to machines, and then *sequencing* the jobs on each machine. We consider the class of problems with additive objective functions that have all jobs in the relevant part of the schedule sequenced according to some priority rule—we refer to it as class (A). The difficult part lies mainly in the assignment of jobs to machines because for a given assignment, we can find the optimal schedule by sequencing the jobs in the relevant part of the schedule according to the priority rule, after which the nonrelevant jobs are scheduled after the other jobs.

Class (A) contains important objective functions such as total weighted completion time, for which the whole schedule is relevant, and objective functions such as the weighted number of tardy jobs and total weighted late work, for which only the on-time part of the schedule is relevant. All these problems are unary $\mathcal{NP}$ -hard but solvable in pseudo-polynomial time for a *fixed* number of machines m by applying the dynamic programming techniques of Rothkopf (1966) and Lawler and Moore

(1969). These pseudo-polynomial algorithms, however, are impractical unless $m = 2$ or the processing times are (very) small.

Additive objective functions pose a computational challenge because it is difficult to compute strong lower bounds. This is nicely witnessed by the research effort that the problem of minimizing the total weighted completion time on m identical parallel machines has attracted since the early days of machine scheduling research; see, for instance, Eastman et al. (1964), Elmaghraby and Park (1974), Barnes and Brennan (1977), Sarin et al. (1988). Using the notation scheme of Graham et al. (1979), we refer to this problem as $P||\sum_{j=1}^n w_j C_j$ or as $Pm||\sum_{j=1}^n w_j C_j$ when the number of machines m is fixed. The lower bounds developed by Webster (1992, 1995), which require pseudo-polynomial time, and Belouadah and Potts (1994) are a big leap forward. Belouadah and Potts also report on the performance of a branch-and-bound algorithm that uses their bound; it is capable of solving instances with up to 20 jobs and 5 machines and 30 jobs on 4 machines.

It is also remarkable that other parallel machine scheduling problems with additive objective functions have not received any attention yet. For instance, no one ventured at the parallel machine problem of minimizing the weighted number of tardy jobs or total late work, although

Subject classifications: Production/scheduling. Sequencing: deterministic. Multiple machine: column generation.

Area of review: MANUFACTURING OPERATIONS.

Operations Research, © 1999 INFORMS
Vol. 47, No. 6, November–December 1999, pp. 862–872

862

0030-364X/99/4706-0862 \$05.00
1526-5463 electronic ISSN

their single-machine counterparts, which are binary $\mathcal{NP}$ -hard, attracted considerable interest; see, for instance, Potts and Van Wassenhove (1988, 1992) and Hariri et al. (1995).

In this paper, we present a methodology that can be used to deal with the problem $P\|\sum_{j=1}^n w_j C_j$ and the other problems in class (A); we describe the approach in detail for the former problem and indicate how it can be modified to deal with the latter problems. The approach is based on formulating the problem $P\|\sum_{j=1}^n w_j C_j$ as a set-covering problem with an exponential number of binary variables, n covering constraints, and a single side constraint. We then solve the linear programming relaxation of this formulation by a column generation approach that uses an $O(n\sum_{j=1}^n p_j)$ algorithm to solve the corresponding pricing problem, where p_j is the processing time of job J_j ($j = 1, \dots, n$). Obviously, if the optimal solution for the linear programming relaxation happens to be integral, then we have identified an optimal solution for the problem $P\|\sum_{j=1}^n w_j C_j$. If not, then we apply a branch-and-bound algorithm to determine an optimal solution. Our computational results show the compelling quality of the linear programming bound, which makes branching often unnecessary, and the superiority of our branch-and-bound algorithm to the algorithms presented before.

When we were conducting this research, Chan et al. (1995) as well as Chen and Powell (1995) independently proposed and analyzed the column generation approach to this formulation of the problem $P\|\sum_{j=1}^n w_j C_j$. Chan et al. emphasized on worst-case performance analysis and probabilistic analysis. They have established two main results. The first one is that the linear programming bound is asymptotically optimal for any number of machines; if $m = 2$, then these values always coincide. Their second main result is that the value of the optimal solution is at most equal to $(1 + \sqrt{2})/2$ times the value of the linear programming bound; this bound is strengthened to 1.04 in case $w_j = p_j$ for all $j = 1, \dots, n$. Chen and Powell show how the formulation can be obtained by Dantzig-Wolfe decomposition. They also propose a branch-and-bound algorithm in which lower bounds are computed by solving the linear programming relaxation through column generation. They do not branch on the completion times, but on the original x_{ij} variables that indicate that job i is processed immediately before job j on some machine. As a consequence, they cannot use the $O(n\sum p_j)$ algorithm to solve the pricing problem after the branching has started, but have to resort to an $O(n^2\sum p_j)$ time algorithm.

This paper is organized as follows. In §2, we present the column generation approach for the problem $P\|\sum_{j=1}^n w_j C_j$. In §3, we describe our branch-and-bound algorithm. In §4, we report on our computational experiments for randomly generated instances of this problem. In §5, we discuss the adaptations of the formulation and the pricing algorithm necessary to apply them to other prominent problems in the class (A). We draw conclusions and point out directions for future research in §6.

2. COLUMN GENERATION FOR $P\|\sum_{j=1}^n w_j C_j$

2.1. Problem Description

There are m identical machines, $M_1, \dots, M_m$, available for processing n independent jobs, $J_1, \dots, J_n$. Job J_i ($i = 1, \dots, n$) has a processing requirement of length p_i and a weight w_i . Each machine is available from time zero onward and can handle no more than one job at a time. Preemption of jobs is not allowed. A *feasible schedule* is a specification of the job completion times $C_1, \dots, C_n$ such that no machine processes more than one job at a time and $C_j - p_j \geq 0$. The objective is to find a schedule with minimum total weighted completion time $\sum_{j=1}^n w_j C_j$. The problem is $\mathcal{NP}$ -hard in the strong sense when the number of machines is part of the problem instance.

We review various basic properties of an optimal schedule that are useful in our column generation approach. First of all, we have the following.

THEOREM 1 (Elmaghraby and Park 1974). *There exists an optimal schedule with the following properties:*

- (i) *The jobs are processed contiguously from time zero onward, and no machine is idle before all jobs have been started.*
- (ii) *The last job on any machine is completed between time $H_{\min} = \sum_{j=1}^n p_j/m - (m-1)p_{\max}/m$ and $H_{\max} = \sum_{j=1}^n p_j/m + (m-1)p_{\max}/m$, where $p_{\max} = \max_{1 \leq j \leq n} p_j$.*
- (iii) *On each machine, the jobs are sequenced in order of nonincreasing ratios w_j/p_j .*
- (iv) *If $w_j \geq w_k$ and $p_j \leq p_k$, then there exists an optimal schedule in which job J_j is started no later than job J_k .*

There exists a dynamic programming algorithm based on the first three observations that runs in $O(n(\sum_{j=1}^n p_j)^{m-1})$ time and space; especially, the space requirement becomes unmanageable when m increases.

In the remainder of this subsection, we derive a time slot for each job in which it needs to be executed. The time slot of each J_j is specified by a release date r_j , before which it cannot be started, and a deadline $\bar{d}_j$, at which it has to be finished. Initially, we have that $r_j = 0$ and $\bar{d}_j = \sum_{k=1}^n p_k/(m + (m-1)p_j/m)$ —the deadlines follow from property (ii) (Belouadah and Potts 1994). Using property (iv), we try to derive tighter release dates and deadlines. Reindex the jobs in order of nonincreasing ratios w_j/p_j ; to avoid trivialities, we assume that $n > m$. We define $\mathcal{P}_j$ as

$$\mathcal{P}_j = \{J_k \mid k < j, w_k \geq w_j, p_k \leq p_j\},$$

and $\mathcal{S}_j$ as

$$\mathcal{S}_j = \{J_k \mid k > j, w_k \leq w_j, p_k \geq p_j\}.$$

Accordingly, there is an optimal schedule in which all jobs in the set $\mathcal{P}_j$ start no later than job J_j . If $|\mathcal{P}_j| > m - 1$, then we may conclude that at least $|\mathcal{P}_j| - m + 1$ jobs belonging to $\mathcal{P}_j$ are completed before the starting time of J_j . Hence, if ρ_j is the sum of processing times of the $|\mathcal{P}_j| - m + 1$ smallest jobs in the set $\mathcal{P}_j$, then the earliest start time of J_j is given by

$$r_j = \lceil \rho_j / m \rceil, \quad (1)$$

where $\lceil x \rceil$ is the smallest integer larger than or equal to x .

In a similar spirit, we try to tighten the deadline of J_j . Because there is an optimal schedule in which J_j starts no later than any job in $\mathcal{S}_j$, we conclude that the amount of work that needs to be done between the start time of J_j and time $H_{\max}$ amounts to at least $\sum_{J_k \in \mathcal{S}_j} p_k + p_j$. Accordingly, we have that J_j can start no later than

$$\delta_j = H_{\max} - \left\lceil \left(\sum_{J_k \in \mathcal{S}_j} p_k + p_j \right) / m \right\rceil. \quad (2)$$

If $\delta_j + p_j < \bar{d}_j$, then we set $\bar{d}_j \leftarrow \delta_j + p_j$ to be the new deadline of J_j .

We summarize these results in the following lemma.

LEMMA 1. *There exists an optimal schedule that satisfies all properties of Theorem 1 and in which each job is executed in the time slot described by the release dates and deadlines derived above.*

2.2. Mathematical Formulation and Column Generation

The $P|\sum_{j=1}^n w_j C_j$ problem belongs to the class (A) because of property (iii), which follows directly from Smith's rule for the single-machine version (Smith 1956). We formulate it as a set-covering problem with an exponential number of binary variables, n covering constraints, and a single side constraint. Define a *machine schedule* as a string of jobs that can be assigned together to any single machine. Let a_{js} be a constant that is equal to 1 if job J_j is included in machine schedule s and 0 otherwise. Accordingly, the column $(a_{1s}, \dots, a_{ns})^T$ represents the jobs in machine schedule s . Let $C_j(s)$ be the completion time of job J_j in s ; $C_j(s)$ is defined only if $a_{js} = 1$. Note that because the jobs in s appear in order of their indices without any idle time between their execution, we have that $C_j(s) = \sum_{k=1}^j a_{ks} p_k$ – remember that we have reindexed the jobs in order of nonincreasing ratios w_j/p_j . Hence, the cost c_s of machine schedule s is readily computed as

$$c_s = \sum_{j=1}^n w_j C_j(s) = \sum_{j=1}^n w_j a_{js} \left[\sum_{k=1}^j a_{ks} p_k \right].$$

We call a machine schedule s *feasible* if $r_j + p_j \leq C_j(s) \leq \bar{d}_j$ for each job J_j included in s . Let S be the set containing all feasible machine schedules. We introduce variables x_s ($s = 1, \dots, |S|$) that assume value 1 if machine schedule s is selected and 0 otherwise. The problem is then to select m machine schedules, one for each machine, such that together they contain each job exactly once and minimize total cost. Mathematically, the problem is then to determine values x_s

that minimize

$$\sum_{s \in S} c_s x_s$$

subject to

$$\sum_{s \in S} x_s = m, \quad (3)$$

$$\sum_{s \in S} a_{js} x_s = 1, \quad \text{for each } j = 1, \dots, n, \quad (4)$$

$$x_s \in \{0, 1\}, \quad \text{for each } s \in S. \quad (5)$$

Condition (3) and the integrality conditions (5) ensure that exactly m machine schedules are selected. Conditions (4) ensure that each job is executed exactly once. Note that the equality sign in the conditions (3) and (4) can be changed into “smaller than” and “larger than,” respectively, without losing the validity of the formulation.

The number of columns involved in this formulation is equal to $\sum_{k=1}^{n-m+1} \binom{n}{k}$. Neither the set-covering problem nor its linear programming relaxation, which is obtained by replacing conditions (5) with the conditions $x_s \geq 0$ for all $s \in S$, can therefore be solved by a method that first generates all feasible columns explicitly. Instead, we resort to an iterative method that considers the feasible columns implicitly: column generation. Starting with a restricted linear programming problem in which only a subset $\bar{S}$ of the variables is available, the column generation method solves the linear programming relaxation of the set-covering formulation by adding new columns that may decrease the solution value, if the optimal solution has not been determined yet; these new columns are not obtained through enumeration, but through a *pricing algorithm* that solves an optimization problem, which is called the *pricing problem*.

The primary issue is the design of the pricing algorithm, which we discuss in the next subsection. Our pricing algorithm is a dynamic programming algorithm that usually generates more than one column with negative reduced cost. We also discuss the issue of which and how many such columns should be added to $\bar{S}$ per iteration.

In §2.4 we design a heuristic to generate the initial set $\bar{S}$. We also discuss an implementation issue like the proper size of the initial set $\bar{S}$. In §2.5 we give a step-wise description of our implementation of the column generation algorithm. In §2.6 we discuss a special type of fractional solution to the linear programming problem.

2.3. The Pricing Algorithm

From the theory of linear programming, we know that a solution to a minimization problem is optimal if the *reduced cost* of each variable is nonnegative. In our problem, the reduced cost c'_s of any machine schedule s is given by

$$c'_s = c_s - \lambda_0 - \sum_{j=1}^n \lambda_j a_{js},$$

where λ_0 is the given value of the dual variable corresponding to condition (3) and $\lambda_1, \dots, \lambda_n$ are the given values of the dual variables corresponding to conditions (4). To test whether the current solution is optimal, we determine if there exists a machine schedule $s \in S$ with negative reduced cost. To that end, we solve the *pricing problem* of finding the machine schedule in S with minimum reduced cost. Because λ_0 is a constant that is included in the reduced cost of each machine schedule, we essentially have to minimize

$$c_s - \sum_{j=1}^n \lambda_j a_{js} = \sum_{j=1}^n \left[w_j \left(\sum_{k=1}^n a_{ks} p_k \right) - \lambda_j \right] a_{js},$$

subject to the release dates and the deadlines of the jobs. Our algorithm, which we call the pricing algorithm, tests whether a feasible solution to the linear programming relaxation is optimal; if the outcome is negative, then it outputs a set of feasible machine schedules s with $c'_s < 0$ among which the machine schedule with minimum reduced cost.

Our pricing algorithm is based on dynamic programming and uses a forward recursion that exploits the property that on each machine the jobs are sequenced in order of increasing indices; recall that we indexed the jobs in order of nonincreasing ratios w_j/p_j . Let $F_j(t)$ denote the minimum reduced cost for all feasible machine schedules that consist of jobs from the set $\{J_1, \dots, J_j\}$ in which the last job is completed at time t . Furthermore, let $P(j) = \sum_{k=1}^j p_k$. For the machine schedule that realizes $F_j(t)$, there are two possibilities: Either J_j is not part of it, or the machine schedule contains J_j . As to the first possibility, we must select the best machine schedule with respect to the first $j-1$ jobs that finishes at time t ; the value of this solution is $F_{j-1}(t)$. The second possibility is feasible only if $r_j + p_j \leq t \leq \bar{d}_j$. If it is, then we add J_j to the best machine schedule for the first $j-1$ jobs that finishes at time $t - p_j$; the value of this solution is $F_{j-1}(t - p_j) + w_j t - \lambda_j$.

The initialization is then

$$F_j(t) = \begin{cases} -\lambda_0, & \text{if } j = 0 \text{ and } t = 0, \\ \infty, & \text{otherwise.} \end{cases}$$

The recursion is then for $j = 1, \dots, n$, $t = 0, \dots, \min\{P(j), \max_{1 \leq k \leq j} \bar{d}_k\}$,

$$F_j(t) = \begin{cases} \min\{F_{j-1}(t), F_{j-1}(t - p_j) + w_j t - \lambda_j\}, & \text{if } r_j + p_j \leq t \leq \bar{d}_j, \\ F_{j-1}(t), & \text{otherwise.} \end{cases} \quad (6)$$

LEMMA 2. *The optimal solution value is then found as*

$$F^* = \min_{H_{\min} \leq t \leq H_{\max}} F_n(t). \quad \square$$

Accordingly, if $F^* \geq 0$, then the current linear programming solution is optimal. If $F^* < 0$, then it is not, and we need to introduce new columns to the problem. Candidates are associated with those t for which $F_n(t) < 0$; they can be found by backtracing.

An important implementation issue is the number of columns to add to the linear program after having solved the pricing algorithm. The more columns we add per iteration, the fewer linear programs we need to solve—but the more columns we add per iteration, the bigger the linear programs become. An empirically good choice appeared to be adding those three columns that correspond to those three values of t for which $F_n(t)$ is most negative.

We further incorporated some speed-ups in the pricing algorithm to reduce the empirical running time. Because these speed-ups are not of interest for the red line of the paper, we have included the details in Appendix A.

Note that the pricing algorithm requires $O(n \sum_{j=1}^n p_j)$ time and space. This means that our column generation approach is not sensible if $m = 2$: The $P2 \parallel \sum_{j=1}^n w_j C_j$ problem is better solved directly through dynamic programming.

2.4. The Randomized List Scheduling Heuristic

We need a set $\bar{S}$ of initial columns to compute the initial dual variables. For this purpose, we use a simple but fast randomized list scheduling heuristic to generate many different solutions for the $P \parallel \sum_{j=1}^n w_j C_j$ problem, followed by an iterative local improvement method for the 10 best solutions.

This randomized list scheduling algorithm generates a schedule by assigning the first $n-1$ jobs, in order of nonincreasing ratios w_j/p_j , randomly to the machines, where an earlier available machine has a larger probability of getting the next job on the list. Specifically, the first available machine has a probability of 80% of getting the next job, the second available machine has a probability of 15%, and the third available machine has a probability of 5%. The last job is always assigned to the earliest available machine.

Such a heuristic works very fast, is expected to give a reasonable solution that satisfies property (iii) of Theorem 1, and is likely to give an alternative solution if it is run again. We therefore run the heuristic between 2,000 and 5,000 times, depending on the size of the instance, thus obtaining many alternative solutions. We store the 10 best solutions and apply neighborhood search to try and improve on each of these solutions. The neighborhood of a feasible schedule consists of all schedules in which the jobs on each machine are sequenced in order of nonincreasing ratios w_j/p_j that can be obtained by the following two types of *changing operations*: moving a job from one machine to another, and swapping two jobs scheduled on two different machines. If there is a better schedule in the neighborhood, then we adopt it as the new schedule. This process is repeated and terminates when no further improvement can be found.

Using this procedure, we obtain a set $\bar{S}$ consisting of $10m$ columns. Note that we cannot guarantee that $r_j + p_j \leq C_j(s) \leq \bar{d}_j$ for each J_j and $s \in \bar{S}$. This is no problem at all—these time slots are derived only to speed up the pricing algorithm.

2.5. Solving the Linear Programming Relaxation

In this subsection, we give a step-wise description of our iterative column generation algorithm to solve the linear programming relaxation.

COLUMN GENERATION ALGORITHM

Step 1. Run the randomized list scheduling heuristic 2,000–5,000 times, store the 10 best solutions, and try to improve these solutions by iterative local improvement. The resulting solutions constitute the initial variable set $\tilde{S}$.

Step 2. Solve the linear programming relaxation to obtain the vector of current dual multipliers λ .

Step 3. Use the pricing algorithm to determine if there are any machine schedules s with negative reduced cost.

Step 4. If such machine schedules exist, then add three machine schedules with most negative reduced cost to the set $\tilde{S}$ —go to Step 2.

Step 5. If no such machine schedule exists, then stop: We have solved the linear programming relaxation to optimality.

Let x^* denote the optimal solution to the linear programming relaxation of the set covering formulation and let S^* denote the set containing all columns s for which $x_s^* > 0$. If x^* is integral, then x^* constitutes an optimal solution for $P \parallel \sum_{j=1}^n w_j C_j$, and we are done. If x^* is fractional, then we need in principle to apply branch-and-bound to close the integrality gap and find an optimal integral solution for our problem, unless x^* is a special type of fractional solution that can be converted without much effort into an integral solution of the same objective value. This special type is discussed in the next subsection.

2.6. A Special Type of Fractional Solution

In this section we discuss a special type of fractional solution to the linear programming relaxation, namely the case in which for each job the completion time is equal in each machine schedule in S^* in which it occurs. This special case is less esoteric than it may seem on first sight—it occurred quite often in our computational experiments. The next result is then a powerful tool to convert such a fractional solution of the linear programming relaxation into an integral solution with the same objective solution. The resulting integral solution is then optimal for the original problem.

THEOREM 2. *If $C_j(s) = C_j$ for each job J_j ($j = 1, \dots, n$) and for each s with $x_s^* > 0$, then the schedule obtained by processing J_j in the time interval $[C_j - p_j, C_j]$ ($j = 1, \dots, n$) is feasible and has minimum cost.*

PROOF. The schedule in which job J_j ($j = 1, \dots, n$) is processed from time $C_j - p_j$ to time C_j is feasible if and only if at most m jobs are processed at the same time and no job starts before time zero. The second condition is obviously satisfied because the C_j values originate from feasible machine schedules. The first constraint is satisfied if we

show that at most m jobs are started at time zero and that the number of jobs started at any point in time $t \in [1, T]$ is no more than the number of jobs completed at that point in time, where T denotes the latest point in time that a job is started. Let $A(t) \subseteq S^*$ be the set of all machine schedules in which at least one job starts at time t ; similarly, let $B(t) \subseteq S^*$ be the set of all machine schedules in which at least one job completes at time t . As $C_j(s) = C_j$, for any machine schedule containing J_j , the number of jobs started at time t is equal to $\sum_{s \in A(t)} x_s^*$; similarly, the number of jobs completed at time t is equal to $\sum_{s \in B(t)} x_s^*$. Because of condition (3), we know that at most m jobs are started at time zero. Because each machine schedule s is constructed such that there is no idle time between the jobs, a job in s can start at time t only if some other job in s is completed at time t . Hence, $A(t) \subseteq B(t)$, which means that the indicated schedule is feasible. It is readily checked that the condition $C_j(s) = C_j$ implies that the cost of this schedule is equal to the cost of the fractional solution, and hence minimal. $\square$

If x^* is fractional and does not satisfy the conditions of Theorem 2, then we need a branch-and-bound algorithm to find an optimal solution. This algorithm is presented in the next section.

3. THE BRANCH-AND-BOUND ALGORITHM

If the optimal solution to the linear program neither is integral nor satisfies the conditions of Theorem 2, then a branch-and-bound algorithm is required to find an optimal solution. From other applications, we know that the branching strategy of fixing a variable at either zero or one does not work in combination with column generation, as the pricing algorithm may come up with this column again, even though we fixed the variable at zero. Our partitioning strategy is based upon splitting the set of possible completion times.

If we have a fractional optimal solution that does not satisfy the conditions of Theorem 2, then there is at least one job J_j for which

$$\sum_{s \in S^*} C_j(s) x_s^* > \min\{C_j(s) \mid x_s^* > 0\};$$

we call such a job J_j a *fractional job*. Our partitioning strategy reflects this property. We design a binary branch-and-bound tree for which in each node we first identify the fractional job with smallest index and, if any, then create two descendant nodes: one for the condition that $C_j \leq \min\{C_j(s) \mid x_s^* > 0\}$ and one for the condition that $C_j \geq \min\{C_j(s) \mid x_s^* > 0\} + 1$. The first condition essentially specifies a new deadline for J_j by which it must be completed, which is smaller than its current deadline $\bar{d}_j$. For the problem corresponding to this descendant node, we therefore set

$$\bar{d}_j \leftarrow \min\{C_j(s) \mid x_s^* > 0\}.$$

Because the jobs in $\mathcal{P}_j$ start no later than J_j and $\bar{d}_j - p_j$ is the latest possible start time of J_j , we must have that each

$J_k \in \mathcal{P}_j$ is completed by time $\bar{d}_j - p_j + p_k$, which may be smaller than its current deadline $\bar{d}_k$. Hence, we let

$$\bar{d}_k \leftarrow \min\{\bar{d}_k, \bar{d}_j - p_j + p_k\}$$

for each $J_k \in \mathcal{P}_j$.

The second condition specifies a release date $\min\{C_j(s) \mid x_s^* > 0\} + 1 - p_j$ before which J_j cannot be started, which is larger than its current release date r_j . For the problem corresponding to this descendant node, we therefore set

$$r_j = \min\{C_j(s) \mid x_s^* > 0\} + 1 - p_j.$$

Because the jobs in $\mathcal{S}_j$ start no earlier than J_j , we can possibly increase the release dates of these jobs as well. For each $J_k \in \mathcal{S}_j$, we let

$$r_k \leftarrow \max\{r_k, r_j\}.$$

Hence, this partitioning strategy reduces not only the feasible scheduling interval of J_j , it may also reduce the feasible scheduling intervals of the jobs in $\mathcal{P}_j$ and $\mathcal{S}_j$.

The nice aspect of this partitioning strategy is that either type of condition can easily be incorporated in the pricing algorithm without increasing its time or space requirement: We can use exactly the same recursion as before.

It may be that the instance that corresponds to our current node does not have a feasible solution; that is, there exists no solution for which $r_j + p_j \leq C_j \leq \bar{d}_j$ for each $j = 1, \dots, n$. The problem of finding out if there exists a feasible solution is, of course, $\mathcal{NP}$ -complete. We therefore check a necessary condition for the existence of a feasible solution. If it fails this check, then we may prune the corresponding node and backtrack.

This necessary condition proceeds by transforming the feasibility problem of determining whether there exists a feasible schedule that meets all release dates and deadlines into an optimization problem. We replace each deadline $\bar{d}_j$ by a due date d_j of equal value and consider the problem of minimizing the maximum lateness $L_{\max}$, which is defined as $\max_{1 \leq j \leq n} (C_j - d_j)$. Because $d_j = \bar{d}_j$, the constraint $C_j \leq \bar{d}_j$ is equivalent to the constraint $C_j - d_j \leq 0$. Hence, the answer to the feasibility problem is “yes” if and only if there exists a feasible schedule with $L_{\max} \leq 0$. The optimization problem is $\mathcal{NP}$ -hard, but several good lower bounds that are quickly computed are known. We use the so-called *set-based* lower bound by Vandevelde et al. (1997), which can be computed in $O(n^2)$ time.

It may also be that the current set of columns $\bar{S}$, which was formed while solving the linear programming relaxation in the previous node, does not constitute a feasible solution because of the new release dates and deadlines. We might use a heuristic to try and generate additional columns that satisfy all release dates and deadlines of the current instance, but there is no guarantee of success because finding even a feasible solution to the problem is an $\mathcal{NP}$ -hard problem. If we would pursue this venue, then if a greedy heuristic does not succeed in finding a feasible solution, we would need to apply an enumerative algorithm.

We work around this problem in the following way. We first remove the infeasible columns that are not part of the current solution to the linear programming relaxation; these columns can be deleted with impunity. As to the infeasible columns that are currently part of the linear programming solution, we increase their costs to some big value, say M , such that the current solution value increases to a value larger than the incumbent upper bound. Using this trick, we can proceed with the column generation algorithm to solve the current instance of the problem because we have ensured that there exists at least one feasible solution.

If we backtrack, then we reduce the costs of these columns to their true values.

4. COMPUTATIONAL EXPERIMENTS

In this section we report on our computational results for the problem $P \parallel \sum_{j=1}^n w_j C_j$. The algorithms were coded in the computer language C; the experiments were conducted on an HP9000/710, which is a 55 mips machine. To solve the linear programs, we used the package CPLEX, version 2.1.

We tested our algorithm on three classes of randomly generated instances:

- (1) Instances with processing times drawn from the uniform distribution $[1, 10]$ and weights from the uniform distribution $[10, 100]$.
- (2) Instances with processing times and weights both drawn from the uniform distribution $[1, 100]$.
- (3) Instances with processing times and weights both drawn from the uniform distribution $[10, 20]$.

First, we report on the performance of our algorithm for instances with $n = 20, 30, 40, 50$ jobs and $m = 3, 4, 5$ machines. Later on, we report our experience with larger instances with up to 100 jobs and 10 machines. For each combination of n , m , and instance class, we have generated 25 instances. As we will see, the performance of our algorithm increases with the number of machines for fixed n , which contrasts with other branch-and-bound algorithms, including the one by Belouadah and Potts (1994). Problems with three machines are the hardest to solve for our algorithm. Recall that problems with $m = 2$ are better solved directly by dynamic programming than by our algorithm.

We divide these instances into “easy” instances, for which no branching was required, and “hard” instances, for which the branch-and-bound algorithm needed to be invoked. For each combination of n and m , we report on the number of easy instances out of 25 and the average time to solve the linear programming problem for them. For the hard instances, we report on the *maximum* number of nodes in the branch-and-bound tree, the *maximum* computation time to solve a hard instance, and the *maximum gap* between linear programming solution value and optimal solution value in absolute terms. The maximum percent excess of the optimal solution value over the

Table 1. Results for randomly generated instances.

n	m	Easy Instances		Hard Instances			ILP = LP
		NB	ACT	MNN	MCT	MGAP	
20	3	24	0.57	2	1.17	0	25
20	4	23	0.41	4	0.78	2	24
20	5	24	0.37	2	0.53	0	25
30	3	17	4.26	2	8.72	1	24
30	4	13	2.52	2	5.73	1	24
30	5	19	1.82	3	2.72	0	25
40	3	7	25.90	10	96.04	1	23
40	4	4	10.41	12	24.83	5	20
40	5	10	6.24	10	14.37	2	22
50	3	2	1.88	14	434.34	1	24
50	4	2	36.83	14	138.61	3	21
50	5	2	14.98	6	46.04	2	24

linear programming solution was always smaller than 0.05% and goes unreported. Finally, we report the number of instances for which the gap is zero.

Tables 1–3 summarize our computational results for the randomly generated instances with up to 50 jobs and 5 machines. The headers of the columns are:

- n = number of jobs;
- m = number of machines;
- NB = number of instances out of 25 for which branching was not required;
- ACT = average computation time in seconds for the easy instances;
- MNN = maximum number of search tree nodes;
- MCT = maximum computation time in seconds for the hard instances;
- MGAP = maximum gap between optimal solution value and lower bound;
- ILP = LP = number of instances out of 25 for which the optimal solution value and lower bound concur.

Table 2. Results for instances with large processing times.

n	m	Easy Instances		Hard Instances			ILP = LP
		NB	ACT	MNN	MCT	MGAP	
20	3	25	0.76	–	–	–	25
20	4	24	0.54	1	0.72	0	25
20	5	24	0.45	4	0.94	4	24
30	3	21	5.22	6	13.07	4	22
30	4	22	3.12	8	6.38	9	24
30	5	22	2.23	8	5.25	10	23
40	3	22	25.71	8	61.26	11	23
40	4	12	10.81	18	55.47	40	16
40	5	17	7.62	12	20.39	14	20
50	3	17	115.48	10	614.05	11	19
50	4	16	44.23	10	90.89	29	20
50	5	14	23.68	16	99.14	15	18

Table 3. Results for instances with homogeneous w_j/p_j ratios.

n	m	Easy Instances		Hard Instances			ILP = LP
		NB	ACT	MNN	MCT	MGAP	
20	3	20	0.54	4	1.45	0	25
20	4	23	0.33	2	0.66	0	25
20	5	19	0.33	2	0.55	0	25
30	3	12	2.95	7	15.77	0	25
30	4	4	1.79	10	10.20	0	25
30	5	10	1.12	18	6.91	2	23
40	3	1	10.39	20	188.28	1	24
40	4	4	4.07	16	64.07	0	25
40	5	3	2.00	18	35.89	0	25
50	3	3	42.35	34	742.82	3	24
50	4	0	–	48	342.82	1	22
50	5	1	6.23	22	184.72	3	24

Table 1 displays our results for instances belonging to class (1), which is used by Belouadah and Potts (1994) as well to test the performance of their branch-and-bound algorithm. They report that their algorithm fails to solve instances within one minute of computation time on a CDC 7600 computer, which is about twice as slow as our machine, if $n = 30$ and $m = 3$ or $m = 4$, and if $n = 20$ and $m = 5$. Note that instances of this size are easily solved by our algorithm.

First of all, Table 1 shows the quality of the linear programming lower bound: Branching is often not required, particularly for the instances with $n \leq 30$, and the lower bound is tight for 281 instances out of 300. Accordingly, if the linear programming solution is not integral, then it is often a question of finding an integral solution of the same value; this is an important reason that the rule stipulated in Theorem 1 is so useful. If branching is required, then we need few nodes only to find and verify an optimal solution. Table 1 confirms the claim that we made earlier: For *fixed* n , the instances grow easier with increasing m . This is plausible: The more machines involved, the fewer jobs we may expect to appear in the optimal machine schedules, and accordingly the smaller the relevant solution space will be. Also note that for $n = 40$ and $n = 50$ the linear programming solution is more often fractional than for $n \leq 30$, although the corresponding value is as often tight. A likely reason for this phenomenon is that the number of optimal fractional solutions grows with n and the chances of “hitting” an integral one decreases accordingly.

Because the pricing algorithm requires pseudo-polynomial time, we may expect that the performance of our algorithm deteriorates with the size of the processing times of the jobs. Table 2 displays our results for instances belonging to class (2), where the processing times are drawn from the uniform distribution $[1, 100]$.

Table 2 indicates that these instances are indeed harder to solve. Not only do we need more time to solve the linear programs, which is entirely attributable to the pricing algorithm, but also slightly more search nodes. The extra time effort is modest, however. Note that the maximum gap is

Table 4. Results for large instances from all three instance classes.

n	m	Easy Instances		Hard Instances			ILP = LP
		NB	ACT	MNN	MCT	MGAP	
60	6	1	35.90	30	128.82	1	20
70	7	1	55.35	22	164.68	2	18
80	8	0	—	38	445.76	7	19
90	9	0	—	26	397.86	3	22
100	10	0	—	30	535.91	3	21
60	6	6	52.65	24	214.62	56	11
70	7	4	110.51	32	321.16	39	9
80	8	7	166.88	32	709.89	38	14
90	9	9	277.26	38	891.25	33	14
100	10	1	490.16	62	1842.66	41	7
60	6	1	13.45	78	482.19	2	21
70	7	0	—	68	663.80	4	19
80	8	0	—	112	1036.77	2	19
90	9	0	—	132	1698.96	5	21
100	10	0	—	142	2211.98	4	14

The top section concerns instance class (1), the middle section instance class (2), and the bottom section class (3).

slightly bigger in comparison with Table 1. As a whole, the results remain satisfactory.

Finally, we may also expect that instances with fairly homogeneous ratios w_j/p_j are hard to solve as well. After all, if all ratios w_j/p_j are close to each other, then the number of relevant columns will be large. This intuition is confirmed by our computational results for the instances belonging to class (3).

Indeed, Table 3 shows that the solution to the linear programming problem is seldom integral, although it is more often tight. Apparently, many optimal solutions are fractional. At the same time, it is more difficult to find an integral solution: We need more search nodes than before. This is quite plausible because when the ratios w_i/p_j are close to each other, the position of a job in the final schedule is not so predetermined, and accordingly our branching rule will be less effective.

Finally, we note that our algorithm is able to deal with instances with $n > 50$ as well, as long as the number of machines is not too small. To give an indication of the performance of our algorithm for such instances, we also report on experiments for instances with 60 jobs on 6 machines, 70 jobs on 7 machines, and so on, up to 100 jobs on 10 machines. The results can be found in Table 4. We see that branching is required for more than 90% of these large instances. In contrast, it is required for less than 50% of the smaller instances. Nonetheless, the lower bound value is still tight for a significant majority of instances. Of course, we need more nodes, the maximum gap between the optimal solution value and the linear programming bound is larger, and we need more computing time—but still, instances of even this size are solvable by our algorithm.

Based on our empirical results, we believe that solving the parallel machine scheduling problem through column generation comes down to cracking the linear programming

Table 5. Performance of the heuristic.

n	m	AGAP	# opt
20	3	0.018	36
20	4	0.037	25
20	5	0.042	20
30	3	0.013	17
30	4	0.038	4
30	5	0.045	11
40	3	0.017	1
40	4	0.029	4
40	5	0.044	3
50	3	0.016	5
50	4	0.028	0
50	5	0.040	1
60	6	0.044	1
70	7	0.051	0
80	8	0.053	0
90	9	0.058	0
100	10	0.059	0

relaxation. Once this has been achieved, only a relatively small number of search nodes are needed to find an optimal solution. This would imply a substantial benefit from faster computers. This is opposite to our experience with most standard branch-and-bound algorithms, which may require an enormous number of nodes.

Finally, we report on the performance of our heuristic. We do not differentiate between instance classes. Table 5 reports for each combination of n and m the average gap (AGAP) between the best solution value found by our heuristic and the optimal solution value; this gap is expressed as a percentage of the latter. Furthermore, it reports for each combination the number of instances out of 75 for which an optimal solution was found. The computing time of the heuristic is negligible.

Our conclusion is that the heuristic performs quite well—as could be expected actually, since the problem has a lot of structure and we used this in the design of the heuristic. The average gap AGAP is never more than 0.06%—the maximum gap that we found was no more than 0.08%. Note that the performance of the heuristic deteriorates if the size of the instances, measured either by the number of jobs or the number of machines, increases.

5. EXTENSIONS

In this section we briefly discuss two other types of problems in the class (A) to which the column generation approach applies:

- problems with additive objective functions for which only a part of the schedule is relevant, such as the weighted number of tardy jobs and total weighted late work;
- parallel machine problems with nonidentical machines.

We indicate the major differences with the approach for the $P||\sum_{j=1}^n w_j C_j$ problem only.

5.1. Complementary Objectives

The weighted number of late jobs and total weighted late work are two important objective functions in which jobs are not penalized as long as they are completed at or before their due dates. These objective functions are defined as follows. A job is called *late* if $C_j > d_j$, where d_j is the due date of J_j . The weighted number of late jobs is denoted by $\sum_{j=1}^n w_j U_j$, where $U_j = 1$ if J_j is late, and $U_j = 0$ otherwise. Total weighted late work is denoted by $\sum_{j=1}^n w_j V_j$ where V_j , the late work of J_j , is defined as the portion of work of J_j that is performed after its due date d_j . Accordingly, we have that $V_j = \min\{p_j, \max\{0, C_j - d_j\}\}$. In both cases, the objective functions are to be minimized.

For either problem, there is an optimal schedule in which each machine first performs the on-time jobs in order of nondecreasing due dates and then the late jobs in any sequence (Potts and Van Wassenhove 1992). The late jobs appear thus in the irrelevant part of the schedule, and in fact it does not matter if, when, and by what machine the late jobs are executed. These problems are therefore equivalent to maximizing $\sum_{j=1}^n w_j(1 - U_j)$, the weighted number of on-time jobs, and $\sum_{j=1}^n w_j(p_j - V_j)$, total weighted on-time work. These problems lend themselves much better for the column generation approach because the pricing algorithm then needs to focus only on the on-time jobs. These complementary problems can then be formulated as maximizing

$$\sum_{s \in S} c_s x_s$$

subject to

$$\sum_{s \in S} a_{js} x_s \leq 1, \quad \text{for each } j = 1, \dots, n, \quad (7)$$

and conditions (3) and (5). Note the sense of condition (7): Machine schedules contain on-time jobs only. The pricing algorithm maximizes the reduced cost, which means that the initialization of the recursion is different; apart from that, the recursion is essentially the same as the one described in §2.3, but a job will be included in a machine schedule only if it is (partially) on time.

5.2. Nonidentical Machines

If the machines are not identical, then the processing time of J_i on M_i is p_{ij} , not p_j , for each i and j . Hence, the cost of a machine schedule then depends on the choice of the machine as well, and we may even have that the priority rule is not equal for all machines. This can be easily overcome by associating a different set of machine schedules to each machine. Let $S(i)$ denote the set of feasible machine schedules for machine M_i ($i = 1, \dots, n$). We need to adjust the formulation given in §2.2 only slightly to accommodate nonidentical machine problems. The problem is formulated

as minimizing

$$\sum_{i=1}^m \sum_{s \in S(i)} c_s x_s$$

subject to

$$\sum_{s \in S(i)} x_s = 1, \quad \text{for each } i = 1, \dots, m, \quad (8)$$

$$\sum_{i=1}^m \sum_{s \in S(i)} a_{js} x_s = 1, \quad \text{for each } j = 1, \dots, n, \quad (9)$$

$$x_s \in \{0, 1\}, \quad \text{for each } s \in S(i), i = 1, \dots, m. \quad (10)$$

For the pricing algorithm, we need to perform the recursion m times, one time for each machine separately. Accordingly, the pricing algorithm runs in $O(n \sum_{i=1}^m \sum_{j=1}^n p_{ij})$ time.

In addition, the partitioning strategy that we proposed in §3 does not apply in case of nonidentical machines. An effective alternative is available, though: Simply use a forward partitioning strategy where jobs are assigned to machines; this partitioning strategy can easily be combined with column generation.

6. CONCLUSION AND FUTURE RESEARCH

Column generation algorithms have been shown to be useful for many intractable combinatorial optimization problems; see for an overview Barnhart et al. (1994). This paper shows that a column generation algorithm is computationally attractive for a certain class of $\mathcal{NP}$ -hard parallel machine scheduling problems as well, including such an important problem as minimizing total weighted completion time.

Our column generation algorithm for this problem proceeds by formulating the parallel machine scheduling problem as an integer linear programming problem with $n+1$ constraints but an exponential number of variables. We solve the linear programming relaxation by column generation, where we use a fast pseudo-polynomial algorithm to compute columns with negative reduced cost, if there are any. Our computational results show that the solution value is at least a singularly strong lower bound on the optimal solution value. Furthermore, the solution is often integral, or fractional but transformable into an equivalent integral solution—in both cases, we have solved the scheduling problem to optimality. If it is not integral, then we need a branch-and-bound algorithm to find an optimal solution. In this case, we have a powerful partitioning strategy to close the gap between lower bound and optimal solution value.

Our computational results show that our algorithm is superior to existing algorithms, which run already into trouble solving randomly generated instances with 20 jobs and 5 machines or 30 jobs and 4 machines. Our algorithm solves

instances of this size in no more than 15 seconds on a 55 mips computer. In fact, we have that the smaller the ratio n/m , the easier the instance is for our algorithm. For instance, we can solve instances up to no more than 50 jobs if $m=3$ —but we can solve instances with up to 100 jobs if $m=10$.

In conclusion, the column generation approach is an effective and powerful solution methodology for parallel machine scheduling problems. We are now investigating if there are any other classes of $\mathcal{NP}$ -hard machine scheduling problems for which this statement holds. Another issue, connected to the problem of minimizing total weighted completion time, concerns the design of a good algorithm for detecting whether a node in the branch-and-bound tree corresponds to a feasible schedule.

ACKNOWLEDGEMENTS

Part of the research for this paper was conducted when the first author visited the Center for Operations Research and Econometrics at the Catholic University of Louvain. The first author was supported by Human Capital and Mobility (HCM), grant number ERBCHBGCT940513. The authors thank an anonymous referee for helpful comments on an earlier draft.

APPENDIX A

IMPLEMENTATION OF THE PRICING ALGORITHM

In the previous work, we took advantage of the property that there is an optimal schedule in which no machine schedule finishes its last job before time $H_{\min}$. Accordingly, we solved the pricing problem by choosing the machine schedule s with smallest $F_n(t)$ value, where $H_{\min} \leq t \leq H_{\max}$. To satisfy this lower bound on t , we may need to add a job J_j to s , although $w_j C_j(s) - \lambda_j > 0$ and we are minimizing.

If we ignore the above property, then we know that there is a machine schedule s with minimum cost in which all jobs J_j have $w_j C_j(s) < \lambda_j$. We exploit this observation to reduce the empirical running time of the pricing algorithm. Define $\Delta_j = \lceil \lambda_j / w_j \rceil$ for each j ($j = 1, \dots, n$); we have that $\Delta_j \geq 1$ because we can discard all jobs J_j with $\lambda_j = 0$. Accordingly, if $\Delta_j \leq \min\{\min_{1 \leq k \leq j} \bar{d}_k, P(j)\}$, then Equation (6) can be replaced by

$$F_j(t) = \begin{cases} \min\{F_{j-1}(t), F_{j-1}(t - p_j) + w_j t - \lambda_j\}, & \text{if } r_j + p_j \leq t < \Delta_j - 1, \\ F_{j-1}(t), & \text{otherwise.} \end{cases}$$

Moreover, the optimal solution value is now found as

$$F^* = \min_{0 \leq t \leq H_{\max}} F_n(t).$$

We like to avoid the explicit computation and storage of all values $F_j(t)$ for $t \geq \Delta_j$ because they are all the same. On the other hand, the recurrence relation needs the

value $F_j(t)$ when computing $F_{j+1}(t)$ and $F_{j+1}(t + p_{j+1})$, so we need a procedure to retrieve the proper value of $F_j(t)$ when it is needed. Note now that for any $t \geq \Delta_j$, we have that $F_j(t) = F_{j-a(j,t)}(t)$, where $j - a(j,t)$ is the index of the last job before J_j that cannot be discarded from the machine schedule with maximum value beforehand; that is, $a(j,t)$ is equal to the smallest value such that $t \leq \Delta_{j-a(j,t)} - 1$. Hence, we know that we did not exclude $F_{j-a(j,t)}(t)$ from the computation. In the same fashion, we have for any $t \geq \Delta_j + p_j$ that $F_j(t - p_j) = F_{j-b(j,t)}(t - p_j)$, where $j - b(j,t)$ is the index of the last job before J_j that may be included in the machine schedule with maximum value. Therefore, $b(j,t)$ is the smallest value such that $t - p_j \leq \Delta_{j-b(j,t)} - 1$. Accordingly, the recurrence relation is then for $j = 1, \dots, n$, $t = 0, \dots, \min\{\Delta_j - 1, P(j), H_{\max}\}$

$$F_j(t) = \min\{F_{j-a(j,t)}(t), F_{j-b(j,t)}(t - p_j) + w_j t - \lambda_j\}. \quad (\text{A.1})$$

In order to make it work and to gain from this adjustment, we need to establish an efficient procedure to find the $a(j,t)$ and $b(j,t)$ values for all j and the appropriate times t . Note we must have $a(j,0) = 1$ for any j ($j = 1, \dots, n$) and $a(j,t) \geq a(j,t-1)$. Hence, when computing $F_j(t)$ we check first if $a(j,t) = a(j,t-1)$; if it is not, then we increase the value $a(j,t)$ in steps of size one until $t \leq \Delta_{j-a(j,t)} - 1$. Accordingly, the computation of $a(j,t)$ requires one check each time we perform computation (A.1) plus $O(n^2)$ operations altogether to find the values $a(j,t)$ if $a(j,t) \neq a(j,t-1)$. We can design a similar procedure to compute the values $b(j,t)$. Hence, the worst-case running time of the pricing algorithm remains the same. The average running time has been reduced, however, because we have restricted the range of the state variable t for which the recursion needs to be performed.

REFERENCES

- Barnes, J.W., J.J. Brennan. 1977. An improved algorithm for scheduling jobs on identical machines. *AIIE Trans.* **9** 25–31.
- Barnhart, C., E.L. Johnson, G.L. Nemhauser, M.W.P. Savelsbergh, P.H. Vance. 1994. Branch-and-price: column generation for solving huge integer programs. Report COC-9403, Georgia Institute of Technology, Atlanta, GA.
- Belouadah, H., C.N. Potts. 1994. Scheduling identical parallel machines to minimize total weighted completion time. *Discrete Appl. Math.* **48** 201–218.
- Chan, L.M.A., P. Kaminsky, A. Muriel, D. Simchi-Levi. 1995. Machine scheduling, linear programming and list scheduling heuristics. Working Paper, Northwestern University, Chicago, IL.
- Chen, Z.L., W.B. Powell. 1995. Solving parallel machine total weighted completion time problems by column generation. Working Paper, Princeton University, Princeton, NJ.
- Eastman, W.L., S. Evan, I.M. Issacs. 1964. Bounds for the optimal scheduling of N jobs on M processors. *Management Sci.* **11** 268–279.
- Elmaghraby, S.E., S.H. Park. 1974. Scheduling jobs on a number of identical machines. *AIIE Trans.* **6** 1–13.

- Graham, R.L., E.L. Lawler, J.K. Lenstra, A.H.G. Rinnooy Kan. 1979. Optimization and approximation in deterministic sequencing and scheduling: a survey. *Ann. Discrete Math.* **5** 287–326.
- Hariri, A.M.A., C.N. Potts, L.N. van Wassenhove. 1995. Single machine scheduling to minimize total weighted late work. *ORSA J. Comput.* **7** 232–242.
- Lawler, E.L., J.M. Moore. 1969. A functional equation and its application to resource allocation and sequencing problems. *Management Sci.* **16** 77–84.
- Potts, C.N., L.N. van Wassenhove. 1988. Algorithms for scheduling a single machine to minimize the weighted number of late jobs. *Management Sci.* **34** 843–858.
- , ———. 1992. Single machine scheduling to minimize total late work. *Oper. Res.* **40** 586–595.
- Rothkopf, M.H. 1996. Scheduling independent tasks on parallel processors. *Management Sci.* **12** 437–447.
- Sarin, S.C., S. Ahn, A.B. Bishop. 1988. An improved branching scheme for the branch-and-bound procedure of scheduling n jobs on m parallel machines to minimize total weighted flowtime. *Internat. J. Production Res.* **26** 1183–1191.
- Smith, W.E. 1956. Various optimizers for single-stage production. *Naval Res. Logis. Quart.* **31** 325–333.
- Vandavelde, A.M.J., J.A. Hoogeveen, C.A.J. Hurkens, J.K. Lenstra. 1997. Lower bounds for the multiprocessor flow shop. Manuscript, Department of Mathematics and Computing Science, Eindhoven University of Technology, The Netherlands.
- Webster, S.T. 1992. New bounds for the identical parallel processor weighted flow time problem. *Management Sci.* **38** 124–136.
- . 1995. Weighted flow time bounds for scheduling identical processors. *Euro. J. Oper. Res.* **80** 103–111.